

Migliorare e mantenere la skill noiza-docs

Per: chi vuole aggiungere componenti, correggere bug o modificare il workflow

Prerequisito: accesso in write al repo `NoizaDev/noiza-pdf-skill`

1. Struttura del repo

Il repo è `NoizaDev/noiza-pdf-skill`, ma va clonato nella cartella `noiza-docs`: è il nome della cartella a decidere come si invoca la skill (`/noiza-docs`).

```
noiza-docs/
├─ SKILL.md           ← istruzioni per Claude (il "cervello" della skill)
├─ README.md          ← guida installazione e uso rapido
├─ guide-utilizzo.md   ← guida per chi genera documenti
├─ guide-contribuzione.md ← questa guida
└─ references/
    ├─ template.html   ← CSS Noiza completo (unica fonte di stile)
    ├─ components.md    ← catalogo componenti con snippet HTML
    └─ logo-noiza_blu.svg ← logo ufficiale per l'header
```

2. Cosa NON modificare senza test

SKILL.md controlla il comportamento di Claude. Una modifica sbagliata cambia come la skill interpreta le richieste per tutti.

CSS in template.html: qualsiasi modifica si riflette su tutti i documenti

rigenerati. Testa sempre su un documento reale prima di fare push.

3. Aggiungere un nuovo componente

Un componente è una classe CSS riutilizzabile. Segui questi 3 passi nell'ordine:

Passo 1: CSS in `references/template.html`

Aggiungi il nuovo stile nella sezione commenti corrispondente:

```
.nome-componente {
  background-color: #f0fdf4;
  border-left: 3px solid #16a34a;
  padding: 10px 14px;
  margin: 10pt 0;
}
.nome-componente p, .nome-componente li {
  font-size: 11pt;
  padding-top: 3pt;
  padding-bottom: 3pt;
}
```

Passo 2: esempio in `references/components.md`

Aggiungi una sezione con: titolo, descrizione breve, snippet HTML pronto all'uso. Segui il formato degli altri componenti nel file.

Passo 3: tabella in `SKILL.md`

Aggiungi una riga alla tabella "Componenti disponibili" con: nome, classe CSS, quando usarlo.

4. Modificare il workflow di generazione PDF

Il workflow è definito in `SKILL.md` (Parte 3) e prevede due vie di export: **Playwright MCP** (preferita) e un **fallback via Chrome DevTools Protocol** quando Playwright non è disponibile. Entrambe partono da un server HTTP locale e condividono la regola sul font Montserrat incorporato in base64.

Attenzione: il workflow è lo stesso per tutti i colleghi. Una modifica sbagliata qui rompe la generazione PDF per chiunque usi la skill. Testa sempre su un documento reale prima di fare push.

5. Colori di riferimento

Nome	Hex	Uso
Blu primario	#2563eb	Bordi, header tabelle, badge, accent
Blu chiaro	#eff6ff	Sfondo key-point
Arancione	#ea580c	Bordo warning
Arancione chiaro	#fff7ed	Sfondo warning
Verde	#16a34a	Badge completato
Grigio testo	#1e293b	Testo principale
Grigio bordi	#e2e8f0	Separatori, bordi tabelle
Grigio sfondo	#f8fafc	Righe alternate, price-box

6. Fare una PR

1. Crea un branch con nome descrittivo: `add-success-box` , `fix-table-overflow`
2. Fai le modifiche nei file coinvolti (CSS, components.md, SKILL.md)
3. Testa generando almeno un PDF completo con il nuovo componente
4. Apri la PR su `main` con una riga di descrizione nel titolo

Commit message consigliato:

```
feat: aggiungi componente .success-box (verde)
fix: correggi overflow tabelle con testo lungo
```

Sorgente e output: i file `.md` sono la sorgente, i file `.html` e `.pdf` sono output rigenerabili. Le guide (utilizzo e contribuzione) fanno eccezione: versioniamo anche i loro `.html` e `.pdf` perché sono la reference che i colleghi leggono. Per i documenti dei progetti cliente, invece, nel repo va solo il `.md`.